Migrating Python AI Prototypes to Cross-Platform Solutions

Tudor Coman
Machine Learning Engineer, Adobe

ML in PL 2025

Agenda

- Introductions (20-25 mins)
- Presentation & setup (30-40 mins)
- Exercise 1 (45 mins)
- 30 minutes break
- Exercises 2-4 (90 mins)
- Q&A, feedback, conclusions (15 minutes)

Objective

- Build a JVM-native recommender system based on embedding similarity
- LLM calls for getting the recommendations in natural language

Introductions

From Python POC to an Enterprise System

- *Why does migration matter before scaling AI?*
  - Most initiatives start in Jupyter notebooks and end up in production
  - There is a big disconnect between data scientists and engineers
  - Factors that can be overlooked: observability, monitoring, throughput, test coverage

The Sweet Spot of Python Prototyping

- Rapid experimentation
- Huge ecosystem (Pandas, NumPy, Langchain etc.)
- Low barrier to entry, even mathematicians know Python nowadays

What can go wrong?

- Dynamic typing: the lack of a strict contract can lead to runtime surprises

```python
def greet(name):
    # expects a string
    print("Hello, " + name + "!")

greet("Alice")   # ✅ Works fine
greet(42)        # ❌ Runtime error: TypeError
```

```python
from typing import List

def sum_numbers(numbers: List[int]) -> int:
    """Expect a list of integers"""
    return sum(numbers)

print(sum_numbers([1, 2, 3]))       # ✅ Works fine
print(sum_numbers(["a", "b", "c"])) # ❌ Runtime error
```

What can go wrong?

- Notebook culture
  - 0 test coverage
  - Hidden state (global variables defined in previous cells)
  - Cell code is generally not idempotent (running a cell multiple times might produce different results)

What can go wrong?

- GIL (Global Interpeter Lock) = prevent race conditions by disabling multi-threading
- No multithreading + interpreted language + REST/RPC overhead = slow app in production
- Dependency management can also be an issue sometimes

Why cross-platform?

- JVM dominates backend stacks in banking, fintech, e-commerce, telecom etc.
- In many cases, you cannot justify just plugging in a Python microservice to do the “AI stuff”
- Production teams also value **maintainability** over **experimentation speed**
- Some JVM benefits: static typing, better dependency management, multithreading
- No Python service = integration with microservices, logging, monitoring, CI/CD which are already in place = no isolation

What is achieved by JVM integration?

- Easy monitoring in enterprise observability stacks
- Native integration with Spring, Kafka, Kubernetes, Prometheus etc.
- Minimal changes to current CI/CD systems

Tooling

ONNX

- Developed by Facebook and Microsoft (Open Neural Network Exchange)
- “The JVM of models” - a runtime that allows models to be loaded in Java, Kotlin, C#, mobile environments etc.
- Compatible with various ML frameworks, such as PyTorch or TensorFlow
- Will help us migrate the inference

Multik

- Numpy equivalent with much better performance
- Will allow us to perform mathematical operations on vectors/tensors
- Disadvantage: works only with Kotlin, which is not as popular as Java

Langchain4j

- Java-native implementation of Langchain
- This will help us run LLM workflows without Python
- Same patterns: chains, prompts, tools, compatibility with multiple LLM providers
- Direct competitor: Spring AI

pgvector

- PostgreSQL extension to store vectors as columns
- Optimised operations for cosine similarity, indexing etc.

Setup

Make sure you have installed the following

- An IDE for Java (I recommend IntelliJ) + JDK 21
- A text editor for Python (VS Code should be fine)
- Git repository: https://github.com/tudorcoman/jvm-ai-workshop

Exercise 1

- During this exercise, we will implement an endpoint that ingests a product in our database
- Once we get the name and description, we want to create an embedding based on name and description and save it in the database
- We will then save the vector in the database and validate this can be performed correctly

Exercise 2

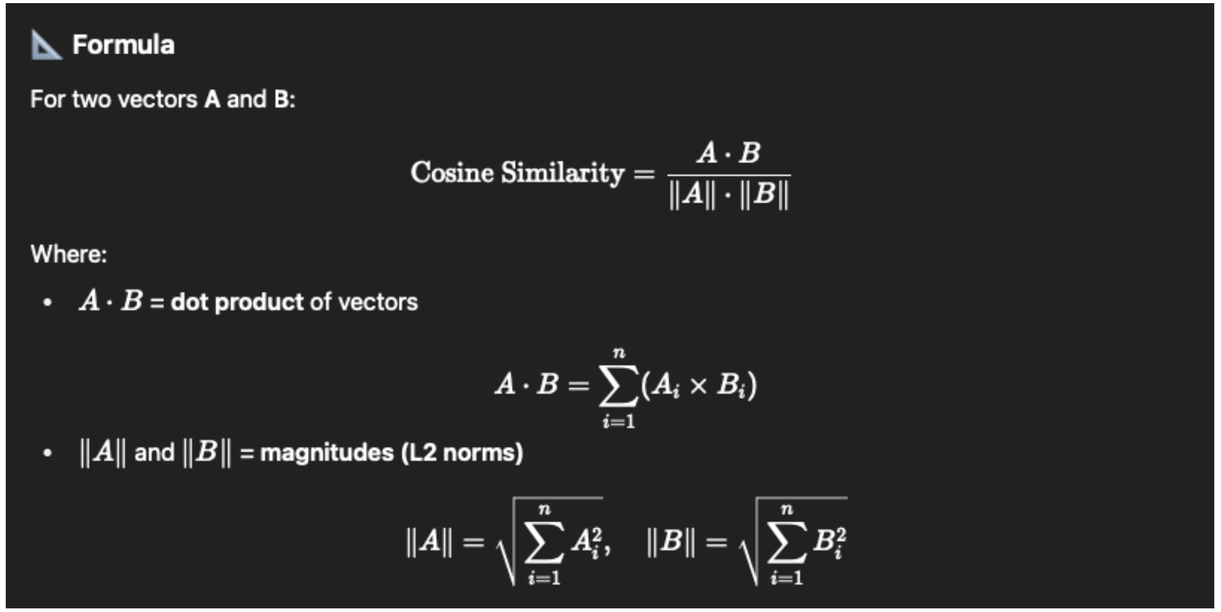
- We will be using an embedding model that is not provided by Langchain4j to generate embeddings from our Java application
- Therefore, we will need to export the model in an ONNX format
- Once exported, we will load it in the Java project to validate it works

Similarity Search

- We are using semantic embeddings for performing similarity search
- For a user's events, we want to perform a weighted average of the product embeddings:
  - VIEWED = 1
  - CLICKED = 2
  - PURCHASED = 3
- This will create a "centroid" where events of higher importance have a higher contribution to the end result
- Then, we can perform cosine similarity on the remaining products to find the ones which would be most appealing to the user

Similarity Search

- Then, we can perform cosine similarity on the remaining products to find the ones which would be most appealing to the user
- Using this approach puts more emphasis on the meaning rather than text richness, length, nuances etc.



Exercise 3

- Perform the similarity search to find recommendations for a user
  - Multik approach
  - **SQL + pgvector approach**
- Send a request to an LLM to get the recommendations in a nice format

Exercise 4

- We need to fill our database with some data
  - 5-10 products
  - 2 users
  - 5-10 events using some of the products

Links

Feedback

LinkedIn